Integrative Industry Synthesis

Kirk Kennedy Lincoln
Woolf University

MS in AI - Masters Capstone Module 7

Integrative Industry Synthesis

Overview and Industry Context

When designing BREH, the main focus was on major LLM providers "farming" tokens and causing inefficiencies within consumer agentic AI systems, as well as lack of efficiency internally. The original intent was to create a self-scaling architecture using periodic inference calls and minimizing down time on agentic tasks. DAGs or Directed Acyclic Graphs are the structure we chose to store agentic tasks for execution, which includes sequencing or single-execution signals. These approaches can ease resource pressure within high-performance compute environments and provide continual task completion without user input.

A future goal is retraining models to help fortify inference around the agent's actions, which could produce a specialized DAG system to "think ahead" and provide graph structures that logically follow the previous user query. A direct application for this type of system is within academic research and SME systems created to aid research professors or R&D departments. In contrast, SME systems leverage several different approaches with one being lexicographical search via directory tree traversal. Although our systems can handle these tasks easily in modern times, we must strive to compress these tools and automate the workflow, which indirectly impacts innovations and progression in technology. Beyond compute cost, repetitive task management has well-documented effects on operator cognitive strain and work performance (Häusser et al., 2014).

Integration Rationale

When reviewing what should be integrated into this project, we had three main concerns: scaling inference, graph generation, and execution policies. The high-level overview of scaling is a slim docker orchestrator to spin up an app image, which will have execution power via smolagents (Hugging Face, n.d.) and LiteLLM . Externally, our system will detect scaling needs in contrast to the container system's metrics, which is a Pickle file generated directly from Module 3. Since BREH runs in Docker, we mapped container CPU utilization stats directly to DUTY (our strongest predictor), and synthesized the remaining feature values to match the Alibaba schema.

During our research, the Gradient Boosting Classifier exceeded other ensemble or regression methods targeting the 75th percentile of relative Alibaba trace data. The approach we took in Module 3 utilized Alibaba's cloud data traces with Scikit-Learn's GradientBoostingClassifier. It is sufficient for demonstration but would need real data to be production ready. The reason we used module 3 directly is the cloud environment most likely applies computing inside containers and has erratic workload spikes in millisecond time, which is overkill for our second-by-second approach.

The original training data had nine features, which were chosen by Alibaba:

- *QPS*: Queries Per Second
- *QUEUE:* Queue Residence Time (how long does something wait in a queue)
- *INFERENCE*: Per-request latency in milliseconds for AI inference
- *DUTY*: Wall-time a worker is active (cpu utilization)
- *BYTES*: Memory or payload footprint
- *PIPELINE:* Parallelism degree within pipeline
- *MODEL*: Model serving the request
- *LORA*: Boolean to detect whether LoRA is used
- *CONTROL*: Queue-Depth, smart router based upon load essentially

Our emphasis was not on scaling docker containers, which is already solved via kubernetes and other infrastructure tools, but utilizing inference to scale microVMs. An original goal was leveraging Lima and

firecracker VMs; however, many operating systems do not allow this KVM usage outside of Linux. Therefore, to save time we implemented docker containers to act like microVMs.

Why emphasize an implementation detail in integration rationale as it is not integral to the project? When BREH is executing graph actions, it's important to note our agentic environments are ephemeral by design, with persistence in a key-value store. Not an original thought by any means, but it leads to the question of whether a persistent agentic environment with memory compacting will lead to optimization, or is it a band-aid used to bridge this gap of time like PowerBuilder was for enterprise software. With the relationship between our "VMs" and the key-value store established, we can expand towards the value added by Module 5's DAG generation approach. Although our results are not directly integrated, the concept and idea are implemented using the LiteLLM lib and using production LLMs to facilitate the DAG generation.

Lastly, Module 6 is directly integrated based upon architecture and implementing the key-value store with etcd, which uses the Raft consensus protocol (Ongaro & Ousterhout, 2014). Currently, this protocol is not critical to our system's success, but it does open the door for distributed deployment and beating out race conditions. Our code is novel and does not use the codebase from project 6; however, our agentic AI approach is essentially identical, which refers back to best practices recommended by the Smolagents library. All in all, each model directly impacts the success seen within our synthesis project. Each Module's idea or results are core pillars in the creation of BREH.

System Design and Technical Decisions

As mentioned previously, our original intent was to apply Firecracker VMs (Agache et al., 2020) but after realizing it is a Linux-only subsystem, we tried bootstrapping it through Lima, only to find out it would not be a viable option. Therefore, we naturally gravitated towards Docker to seat our execution agent using container injections (Merkel, 2014). One important note before explaining the architecture: the research we conducted in Module 2 placed LLM latency as a huge strain on computing environments, with a Spearman correlation of 0.924 between local execution time and total latency; therefore, the LLM call is blocking. With that finding established, the architecture follows.

Let's assess a bottom-up approach, which can ease any mental strain when trying to connect the dots. There are two main models created to enforce Pydantic typing: Graph and Step. Each Graph is a composition of Steps, a step is weighted, contains a tool call and organized dependency (designates position in execution chain, can we run in parallel?). A Graph contains a list of these steps, which is created by and supplied to the PlannerAgent (refer to documentation). The PlannerAgent and ExecutorAgent possess a scaling inference tool named ScalingPredictorTool, which helps infer scaling needs based upon computing metrics.

Originally, we allowed the LLM to tell us when we should scale out graph steps on initial executions. Useful, since it doesn't cause much overhead and we are going to the LLM regardless of our initial call. The ExecutorAgent contains a second external aggregate named DockerMetricSource; in contrast, our ExecutorAgent has the ability to "override" the LLM recommendations when it comes to scaling needs. Therefore, we can receive the info of "scale out graph steps for this because it's a large task", which helps bypass bottlenecks due to computation and avoids unnecessary inference calls to the Module 3 model. It should be noted that improvements to this area are needed moving forward and should be listed as a priority when continuing development. LLM generated data for scaling, bridges the gap for future innovations in the "Small Language Model" space such as: Phi-3, Gemma, Llama 3, where querying compute internals would be considered cheap or even relatively free in terms of cyclomatic complexity.

When thinking about our transport layer, we chose gRPC since it's the easiest technology to bridge concurrent system programming (Golang) and ML-centric architectures (Python, Pytorch, smolagents). There are two files, main.py and bench.py, which inherently perform the same tasks, just the latter is considered automated integration testing. Much of the Python system is housekeeping (logging, tool call orchestration, facilitating DAG generation, etc.) so the Golang side is where the actual runner and agentic injection happens. The file agent_entrypoint.py is copied into every docker container and speaks gRPC in order to communicate to the GraphStore, which speaks directly to etcd. All of our gRPC protos are created in order to interact with this "sterilized" agent environment. Although we have insecure channels speaking with the GraphStore stub, the general protobuf security can be implemented to these connections.

With this picture in mind, our main driving force was creating a sterile agentic-environment to perform A2A (Agent 2 Agent) interaction, which mimics a SYN/ACK relationship with 3-way handshakes. When creating agentic systems, we want to hide certain context and data from certain agents, particularly external agents. What logically follows is an agentic AI card repository, which allows the spawned docker container (or microVM) to pull in a third-party agent to complete a specific task. Since our graphs can be split between containers, tracing data and reassembling received data will be insurmountably more difficult than a simple TLS connection to a company's servers.

Ethical Considerations and Responsible AI

The ethical considerations in our system are the lack of linear tracing within the agentic system. Since our steps are being executed apart from a single smolagents runtime, we do not have the singular "thought trace", which could violate XAI considerations. In BREH's defense, that implementation is not an R&D project but merely a software implementation that we felt was not necessary due to the goals of our research.

In contrast, our system truly answers "how do we enforce HIPAA, PII anonymity, with agentic AI?" (U.S. Department of Health and Human Services, 2013). When referring to cohomology, there is the

concept of a local-to-global gluing obstruction: given a cover of a space by open sets, sections defined locally on each piece may or may not glue into a single global section, with the obstruction captured by a class in $\check{H}^1$ of the cover with values in the relevant sheaf (Hartshorne, 1977). When that obstruction is nontrivial, no global section exists; the local data simply cannot be reassembled. BREH's privacy architecture mirrors this picture and provides a nontrivial obstruction using a partitioned compute environment for agentic AI. The partition of graph steps into separate containers is considered our "cover". Moreover, we do not understand the bounds of each and every prompt result, so we will consider agentic behavior an "open set". The "glue" would be malicious reconstruction of PII, business operations, or other high-level information that is considered private IP.

Given a sheaf F on space X, the global section functor $\Gamma(X, F)$ is left-exact but usually not surjective, and local sections failing to hoist into a global section is what $\check{H}^1$ captures. Since we can associate each homology (graph step) between the overarching "agentic task", then our mapping of results is considered surjective but reconstruction is mathematically proven to mostly fail. The privacy enforcement is to consider peripheral/edged graph steps MUST be executed by non-affiliated agents. Moving into the future, our policies will not be solely driven by data retention and retrieval, but how efficiently we can execute a single task among many non-connected agents. BREH does not provide cryptographical guarantees or any sort of differentiable and/or elliptical security measures in terms of transport. The main focus is on pure data being siphoned off by agents and our previous analysis is meant to model how group theory can prove our data is most likely non-reconstructable by overtly hostile actors.

Lastly, during our research we found Anthropic's AI to be "farming" tokens or using excessive resources to accomplish medium-type tasks (web research and synthesis). When reviewing our results.csv file (see /data in BREH repo), our agent used 7255 input tokens and 6050 output tokens, which total to a cost of $0.0375 versus $0.113619. In regards to criticism of selection bias in our results, our first sample question "Building a J2EE server" resulted in costs of BREH: $0.008057 and LiteLLM + Smolagents + Anthropic: $0.031095, which is almost a four-to-one ratio in favor of BREH, where the question is considered a "trivial web-search tool-calling test task".

Evaluation and Reflection

As previously mentioned, Firecracker VM was not a viable option and required us to pivot into using docker containers. Hopefully in the future, microVMs will become more accessible across all operating systems and provide the necessary foundation for BREH to be highly successful in production business environments. An intended end goal for BREH in its entirety is the orchestration of "returning visits" for agentic AI. Our execution environments and policies are ephemeral and terminal in nature by default. One concept we implicitly attacked with this approach I'll call Authorization Torsions.

Imagine you own a business. Now picture a manager that you caught stealing, you would want them fired immediately. However, before being caught, the manager hired their friend and trained them to become a manager. Without knowing this, you promote that employee to replace the fired manager, who can then turn around and perform the same malicious actions.

If agents somehow "stick around" in the corners and perform circular authorizations, then an "open door" policy implicitly applies to your production environment. The agent would be malicious in breaking the rules and creating havoc within your server; however, it would have permission to be there while executing those overtly hostile actions. Therefore, data persists; agents do not, and that is the default nature. Eventually, one striving goal is to implement a DSL, which would contain symbolic logic and directives in order to facilitate "elastic memory" between agentic sessions and only provide what the agent needs to know. We did not believe that engineering a "smart cache" captured the full opportunity of applying agentic AI to business computers. Contrastingly, this type of cache would in itself need to be partitioned across computing environments to maintain a "cover" of protection against PII leakage. Here comes the argument for blockchains, specifically private (e.g., Hyperledger Fabric), which decentralizes the location of PII data and removes structured, self-referential data from the caching environment altogether.

Professional Relevance

In a world where AI seems more like a money pit than an innovative addition to technical infrastructure, BREH presents a unique opportunity to satisfy the AI-accelerated workflows but manage the premium cost overhead. Pairing this idea with newer technologies such as Rust, we can begin to see how the efficiencies would accelerate AI usage among organizations. Firecracker VM would be hugely useful; however, maybe a desktop application with a built-in "memory arena" can quell the necessity for microVM usage onboard.

As cloud-computing and computing in general ascend the ladder of parallelism using the rungs of concurrency, utilizing blockchain technologies and other fault-tolerant, free-read technologies will become hugely necessary. Dynamically generating DAGs to execute before they are needed sidesteps the largest problem with agents to date: round-trip LLM latency waits. The amount of time and engineering that needs to take place in order to moderate the agentic traffic for a business environment is not worthwhile unless the company is yielding millions to billions from it. How does a small business owner leverage agents in a cost-effective fashion? Through predictive DAG generation.

Companies currently store "business processes"; however, a business rules engine is brittle and takes years to create a battle-tested engine, which can smartly execute business needs. The vision of BREH is creating a "git"-like environment where DAGs are being generated cheaply if not considered free (compute-wise), and fed directly to a pool of agents. The Graphs will be judged and assessed then executed while dropping the remaining branches. Essentially, most jobs are repetitive and considered a "closed set" of what needs to be produced, so our mapping vector is tightened, yet strengthened, since our outcomes will be (for the most part) considered deterministic. Throughout our research, malicious agentic prompt injection or future vulnerabilities will produce "out-of-scope" results or data leakage. BREH reinforces this security by only providing what the agent needs to see while obfuscating the rest of the data. Since etcd is not ephemeral, future engineering design can produce a "return visit" state hydration within a newly created sanitary agentic environment.

References

Agache, A., Brooker, M., Florescu, A., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., & Popa, D.-M. (2020). Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)* (pp. 419–434). USENIX Association.

Hartshorne, R. (1977). *Algebraic geometry*. Springer-Verlag.

Häusser, J. A., Schulz-Hardt, S., Schultze, T., Tomaschek, A., & Mojzisch, A. (2014). Experimental evidence for the effects of task repetitiveness on mental strain and objective work performance. *Journal of Organizational Behavior, 35*(5), 705–721. https://doi.org/10.1002/job.1920

Hugging Face. (n.d.). *smolagents documentation*. Retrieved from https://huggingface.co/docs/smolagents

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239), Article 2.

Ongaro, D., & Ousterhout, J. (2014). In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)* (pp. 305–319). USENIX Association.

U.S. Department of Health and Human Services. (2013). *Summary of the HIPAA Privacy Rule*. https://www.hhs.gov/hipaa/for-professionals/privacy/laws-regulations/index.html